

Pipeline de données temps réel pour Nvidia et dashboard analytique

Kaddouri Oussama

Oussama Laghchim

Anouar Tayi

Abstract—Ce rapport présente un pipeline de données en temps réel et un dashboard analytique dédiés à Nvidia (NVDA). Le système ingère des données de marché et des données sociales issues de plusieurs sources (Yahoo Finance, Reddit, StockTwits, Yahoo News) via des producteurs Apache Kafka, traite les streams avec Apache Spark Structured Streaming (schema validation, timestamp handling) et persiste les jeux de données préparés dans MongoDB Atlas. Un dashboard Streamlit fournit des KPI, des visualisations techniques (candlesticks, moving averages, volatility) et des analyses de sentiment. La solution est conteneurisée avec Docker Compose pour une orchestration reproductible (Zookeeper, Kafka, Spark et services applicatifs). Nous décrivons l'architecture, la configuration des topics et des schemas, la logique de processing, l'organisation des collections et les modalités d'utilisation.

Index Terms—Kafka, Spark Structured Streaming, MongoDB Atlas, Streamlit, Analyse boursière, Reddit, StockTwits, Yahoo Finance, Nvidia (NVDA), Pipeline temps réel

I. INTRODUCTION

Nvidia (NVDA) est un titre fortement suivi dont la volatilité attire autant les investisseurs que les communautés en ligne. L'objectif de ce projet est de concevoir un pipeline de données en temps réel capable de collecter, traiter et visualiser des informations de marché et sociales afin d'aider à l'analyse et à la prise de décision.

Nous proposons une architecture moderne basée sur Apache Kafka (ingestion et transport), Apache Spark Structured Streaming (stream processing), MongoDB Atlas (stockage persistant) et Streamlit (visualisation). Le déploiement local reproductible s'appuie sur Docker Compose pour orchestrer Zookeeper, Kafka et les services de l'application.

Les contributions principales sont les suivantes :

- Conception d'un pipeline temps réel multi-sources (marché + réseaux sociaux) autour d'un même actif.
- Implémentation d'un Spark Streaming processor avec schemas explicites et écriture en continu vers MongoDB.
- Dashboard Streamlit pour l'exploration interactive (KPI, visualisations, activité sociale).

II. CONTEXTE, OBJECTIFS ET PÉRIMÈTRE

A. Problématique

Les analystes et investisseurs combinent souvent des signaux hétérogènes : séries de prix (OHLCV) et signaux sociaux (posts, news) susceptibles d'expliquer ou d'anticiper des mouvements. Le défi technique est de rendre ces données disponibles de manière homogène, quasi temps réel, et consultables via un dashboard.

B. Objectifs

Les objectifs principaux du projet sont :

- Mettre en place une architecture event-driven basée sur Kafka pour l'ingestion multi-sources.
- Assurer un stream processing robuste avec Spark Structured Streaming (schema validation, timestamp handling, checkpointing).
- Stocker les données préparées dans MongoDB Atlas avec un data model simple et exploitable.
- Fournir un dashboard (Streamlit) permettant l'exploration des KPI et des visualisations techniques.

C. Périmètre

Le périmètre couvre la collecte et l'analyse autour de NVDA : données de prix, posts Reddit, messages StockTwits et news Yahoo. Le projet ne vise pas (à ce stade) la prédiction automatique ni un modèle de trading, mais fournit une base solide pour ces extensions.

Le dépôt contient toutefois un module expérimental de séries temporelles (forecasting) qui illustre comment exploiter les données stockées dans MongoDB pour effectuer une prévision courte. Cette partie est exécutée hors-ligne (offline) et n'est pas encore intégrée au pipeline temps réel ni au dashboard.

III. SPÉCIFICATIONS ET CONTRAINTES

A. Exigences fonctionnelles

- Ingestion continue des événements (JSON) par source dans des topics Kafka dédiés.
- Stream processing pour parser, typer, enrichir et persister en continu.
- Dashboard interactif : filtres temporels, KPI, vues prix/volatilité/volume, vues sociales.

B. Exigences non fonctionnelles

- **Reproductibilité** : déploiement local par Docker Compose.
- **Scalability** : possibilité d'ajouter des topics, partitions, nouveaux producteurs.
- **Robustesse** : tolérance aux redémarrages via checkpoint-Location et séparation producteurs/consumers.
- **Traçabilité** : logs et organisation des streams par collection.

IV. ARCHITECTURE DU SYSTÈME

L'architecture suit un schéma en quatre couches : (1) **Producers** (scripts Python) qui publient les événements dans Kafka ; (2) **Kafka** qui expose des topics pour chaque source ; (3) **Spark Structured Streaming** qui lit, valide, enrichit et normalise les streams ; (4) **MongoDB Atlas** qui stocke les collections prêtes à l'analyse, consommées par le **dashboard Streamlit**.

Les principaux composants du dépôt :

- **scripts/** : scrapers et producers (Yahoo Finance, Reddit, StockTwits, Yahoo News).
- **src/processors/spark_streaming.py** : logique de stream processing avec schemas explicites.
- **src/config/** : configuration Kafka et MongoDB.
- **Dashboard Streamlit** : interface analytique (KPI et visualisations) basée sur les collections MongoDB.
- **docker-compose.yml** : orchestration locale (Zookeeper, Kafka).

V. CHOIX TECHNOLOGIQUES

A. Kafka

Kafka assure le découplage entre la collecte (producers) et la consommation (Spark). Les topics rendent l'architecture extensible : une nouvelle source de données se traduit par un nouveau producer et un nouveau topic.

B. Spark Structured Streaming

Spark Structured Streaming fournit un modèle déclaratif (DataFrame) adapté aux transformations et à la schema validation. Dans ce projet, le processor lit des messages Kafka, parse le JSON, ajoute un champ temporel unifié et écrit les événements en mode append.

C. MongoDB Atlas

MongoDB Atlas est utilisé comme couche de persistance et de requête, pratique pour stocker des documents hétérogènes (prix, posts, news) et alimenter un dashboard.

D. Streamlit

Streamlit permet de construire rapidement une interface interactive orientée analyse (graphiques, filtres, KPI) sans nécessiter une pile front-end complexe.

VI. SOURCES DE DONNÉES

Nous ingérons quatre streams principaux :

- **Yahoo Finance** : prix historiques et quotidiens (Open, High, Low, Close, Volume, Dividends).
- **Reddit** : posts relatifs à Nvidia, incluant titre, auteur, score et date.
- **StockTwits** : messages financiers autour de NVDA.
- **Yahoo News** : actualités financières.

Chaque source est publiée dans un *topic* Kafka dédié et transformée en une collection MongoDB correspondante (cf. Tableau V).

A. Format des événements

Les producers publient des messages au format JSON dans Kafka. Le Spark processor applique un `StructType` adapté à chaque topic afin d'éviter les erreurs de typage et de garantir des champs stables en base.

VII. DATA MODEL ET SCHEMAS

Cette section décrit le data model côté Kafka/MongoDB. Les événements sont sérialisés en JSON puis reconstitués côté Spark via un schema explicite. Les documents MongoDB suivent la même structure que les événements, avec l'ajout d'un champ `timestamp` (ingestion timestamp ou conversion depuis un champ `date` source).

A. Schemas par stream

TABLE I
SCHEMA (RÉSUMÉ) : TOPIC_STOCK_PRICES

Field	Type
Date	string
Open/High/Low/Close	double
Volume	long
Dividends/Stock Splits	double
timestamp	timestamp

TABLE II
SCHEMA (RÉSUMÉ) : TOPIC_REDDIT_POSTS

Field	Type
date	string
titre/texte/subreddit	string
score/commentaires	int
timestamp	timestamp

TABLE III
SCHEMA (RÉSUMÉ) : TOPIC_STOCKTWITS_POSTS

Field	Type
id	long
date/user/text/sentiment	string
likes	int
timestamp	timestamp

TABLE IV
SCHEMA (RÉSUMÉ) : TOPIC_YAHOO_NEWS

Field	Type
publisher/title/link/summary	string
pubdate	string
timestamp	timestamp

B. Conventions de nommage

Les topics suivent le pattern `topic_{source}` et les collections MongoDB suivent des noms explicites (ex. `stock_prices`, `nvidia_reddit_posts`). Cette convention simplifie l'opérationnel : on peut associer directement un consumer/stream à une collection.

VIII. INGESTION ET TRANSPORT (KAFKA)

Kafka joue le rôle de bus d'événements. Le fichier `docker-compose.yml` expose deux endpoints : `localhost:9092` pour les producteurs côté machine hôte et `kafka:29092` pour les services conteneurisés. Le processeur Spark consomme `Kafka:29092`.

Les topics principaux (un par source) sont listés dans le Tableau V. Les producers publient typiquement sur le dernier offset (consommation `startingOffsets=latest`).

A. Rôle de Kafka dans le découplage

Kafka permet un découplage fort entre ingestion et processing :

- Les producers peuvent publier à leur rythme (API limits, scraping rate) sans dépendre du consumer.
- Spark peut redémarrer sans perdre l'historique récent (selon la retention Kafka).
- L'ajout d'un nouveau consumer (par ex. un module de sentiment analysis) ne nécessite pas de modifier les producers.

B. Topics, partitions et retention (principes)

Dans un contexte production, on configure typiquement : (i) **partitions** pour augmenter le throughput, (ii) **replication factor** pour la tolérance aux pannes, (iii) **retention** pour contrôler l'historique disponible. Dans ce projet, la configuration est volontairement simple pour un déploiement local.

IX. STREAM PROCESSING (SPARK)

Le fichier `src/processors/spark_streaming.py` applique des schemas stricts (`StructType`) et gère les champs de date/heure pour fiabiliser l'ingestion :

- Association

<code>topic</code>	<code>-i</code>	<code>collection</code>
:	<code>topic_stock_prices</code>	<code>-i</code>
	<code>stock_prices, topic_reddit_posts</code>	<code>-i</code>
	<code>nvidia_reddit_posts, etc.</code>	
- Validation des types (`DoubleType`, `LongType`, `StringType`) et normalisation des dates (par ex. champ `Date`).
- Ajout d'un ingestion timestamp

<code>(current_timestamp())</code>	et	enrichissements
		basiques.

A. Processing steps

Pour chaque configuration de stream : (1) **Lecture Kafka** via `readStream.format("kafka")`. (2) **Parsing JSON** en convertissant `value` en chaîne puis `from_json(...)`. (3) **Timestamp handling** : conversion du champ `date` en `timestamp` (`to_timestamp`) si disponible, sinon usage de `current_timestamp()`. (4) **Écriture MongoDB** via le connecteur MongoDB en mode `append` avec un `checkpointLocation` dédié par collection.

B. Micro-batch execution model

Spark Structured Streaming exécute le stream en micro-batches. Chaque micro-batch lit un intervalle d'offsets Kafka, applique les transformations `DataFrame`, puis déclenche l'écriture. Ce modèle simplifie la reprise après incident : Spark peut relire depuis les offsets stockés dans le checkpoint.

C. Fault tolerance et checkpoint

Le paramètre `checkpointLocation` (un dossier par collection) stocke l'état nécessaire au redémarrage (offsets Kafka, metadata de streaming). Cela permet :

- Redémarrage du job sans réinitialiser l'état.
- Protection contre les doublons dans certaines conditions (selon les sinks et le traitement).

Dans un environnement Windows/local, le chemin `/tmp/checkpoints/...` peut être adapté selon l'OS (ou monté via Docker). Le principe reste identique.

D. Gestion des dates

Les sources n'ont pas toutes le même format de date (ex. `Date`, `date`, `pubdate`). Le processor unifie ces champs en un `timestamp` utilisé pour le tri et l'analyse. Une amélioration possible est l'usage de formats explicites (`to_timestamp(col, format)`) si certaines sources varient.

X. STOCKAGE (MONGODB ATLAS)

Les données sont persistées dans la base `NvidiaStockData` sur MongoDB Atlas. Les collections principales : `stock_prices`, `nvidia_reddit_posts`, `nvidia_stocktwits_posts`, `nvidia_yahoo_news`. L'utilisation de schemas stables facilite les requêtes analytiques et l'alimentation du dashboard.

A. Indexing et requêtes (recommandations)

Pour accélérer les requêtes dashboard, on recommande des indexes sur `timestamp` et, selon les besoins, sur des champs comme `subreddit` ou `publisher`. Dans ce projet, les indexes peuvent être créés côté Atlas selon les volumes.

XI. TABLEAU DE BORD (STREAMLIT)

Le dashboard Streamlit expose des KPI clés (prix de clôture moyen, volume moyen, plus haut/bas historiques) et des visualisations professionnelles (courbe du cours, chandeliers, moyennes mobiles, volatilité, volumes). Les données sont lues depuis MongoDB et agrégées pour des analyses interactives.

A. Organisation du dashboard

Le dashboard est structuré en vues permettant de séparer l'analyse marché et l'analyse sociale :

- **Tendances prix** : Close over time, candlesticks, moving average.
- **Volume et volatilité** : volume bars, volatility index (High-Low).
- **Social streams** : Reddit/StockTwits feed, agrégations simples (fréquence, score/likes).

B. KPI et interprétation

Les KPI principaux fournissent des repères rapides : *average close* (niveau moyen), *average volume* (liquidité), *all-time high/low* (zones psychologiques). Ces indicateurs sont utilisés comme résumé avant une analyse plus détaillée via les charts.

10. 🌟 Indicateurs clés (KPI)

Prix de clôture moyen	Volume moyen	Plus haut historique	Plus bas historique
\$156.30	216,414,487	\$212.18	\$86.60

1. 📈 Courbe du prix de clôture ➡

Évolution du prix de clôture



Prix de clôture avec moyenne mobile 7 jours



Prix de clôture avec moyenne mobile 7 jours



XII. DÉPLOIEMENT ET ORCHESTRATION

Le déploiement local s'appuie sur `docker-compose.yml`. Kafka est exposé à la fois en `localhost:9092` et en interne via `kafka:29092` (`KAFKA_ADVERTISED_LISTENERS, PLAINTEXT_INTERNAL`). Spark consomme l'URL interne `Kafka:29092`, ce qui simplifie l'intégration avec les conteneurs.

XIII. MODE OPÉRATEUR (EXÉCUTION)

L'exécution typique suit trois étapes :

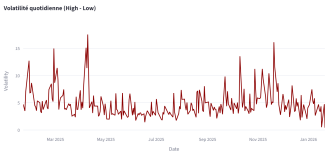
- **Démarrer l'infrastructure** : lancer Docker Compose pour Zookeeper et Kafka.
- **Démarrer les producteurs** : exécuter le script `start_producers.bat` (ou scripts Python équivalents) afin d'alimenter Kafka.



2. 📊 Volume échangé quotidien



9. 📊 Indice de volatilité (High - Low)



- **Démarrer le processing et le dashboard** : lancer le Spark Streaming processor puis le dashboard Streamlit.

XIV. SÉCURITÉ ET BONNES PRATIQUES

Dans une version de production, les identifiants (MongoDB, API keys, etc.) ne doivent pas être codés en dur. Ils doivent être fournis via variables d'environnement, fichiers `.env` (non versionnés) ou un gestionnaire de secrets. La rotation des clés et la limitation des droits côté base sont recommandées.

XV. RÉSULTATS ET INDICATEURS

Le dashboard permet d'explorer :

- **KPI** : prix de clôture moyen, volume moyen, plus haut/bas historiques.
- **Tendances** : chandeliers OHLC, moyennes mobiles, volatilité (High-Low).
- **Activité sociale** : agrégations basiques des posts Reddit et StockTwits.

Ces indicateurs offrent une vue synthétique utile pour l'analyse technique et contextuelle.

XVI. SYSTÈME D'ANALYSE FINANCIÈRE INTELLIGENTE

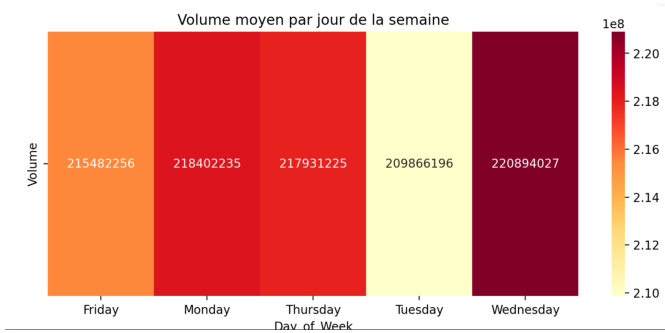
A. Architecture du Système

Le système d'analyse financière intelligente constitue la couche décisionnelle finale du pipeline, intégrant l'ensemble des données et prédictions pour générer des recommandations de trading actionnables. L'architecture repose sur trois composants principaux interconnectés :

- **Module d'Orchestration** : Coordonne l'exécution séquentielle des scripts d'analyse fondamentale et de prévision

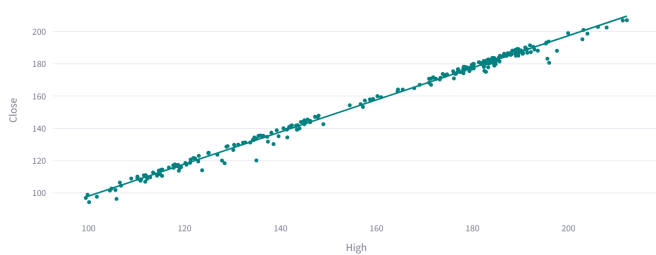
TABLE V
CORRESPONDANCE DES TOPICS KAFKA ET COLLECTIONS MONGODB

Topic Kafka	Collection MongoDB
topic_stock_prices	stock_prices
topic_reddit_posts	nvidia_reddit_posts
topic_stocktwits_posts	nvidia_stocktwits_posts
topic_yahoo_news	nvidia_yahoo_news



8. ● Corrélation High vs Close

Comparison High vs Close



- **Moteur d'Analyse par LLM** : Exploite un modèle de langage avancé (Groq LLaMA 3.3 70B) pour l'interprétation multi-modale des données
- **Système de Distribution** : Automatise la génération et l'envoi des rapports via Telegram

B. Pipeline d'Exécution Automatisé

1) *Orchestration des Composants*: Le pipeline implémente une architecture à trois phases avec validation intermédiaire :

1) Phase 1 : Analyse Fondamentale

- Exécution du script `fundamental_analysis.py`
- Extraction et agrégation des métriques de sentiment social
- Génération du fichier `nvidia_fundamental_features.csv`
- Timeout de sécurité : 10 minutes

2) Phase 2 : Prédiction des Prix

- Exécution du script `stock_forecasting.py`
- Calibration des modèles Prophet et XGBoost
- Génération des prédictions ensemble sur 3 jours
- Production du fichier `tuning_results.json`

3) Phase 3 : Analyse par IA

- Chargement des données historiques (14 derniers jours)
- Consolidation des métriques fondamentales
- Invocation du modèle de langage pour recommandation

- Génération du rapport au format PDF

2) *Gestion des Erreurs et Robustesse*: Le système implémente plusieurs mécanismes de fiabilité :

```
def verify_output_files():
    """Verifie l'existence et la fraicheur des fichiers"""
    files_to_check = [
        (FUNDAMENTAL_OUTPUT, "Fundamental Analysis CSV"),
        (FORECASTING_OUTPUT, "Forecasting Results JSON")
    ]

    all_valid = True
    issues = []

    for file_path, file_name in files_to_check:
        if not Path(file_path).exists():
            issues.append(f"Missing: {file_name}")
            all_valid = False
        else:
            file_time = datetime.fromtimestamp(
                Path(file_path).stat().st_mtime
            )
            age_hours = (datetime.now() - file_time).total_seconds()
            / 3600

            if age_hours >= 24:
                issues.append(
                    f"Warning: {file_name} is {age_hours:.1f}h old"
                )

    return all_valid, issues
```

Listing 1. Mécanisme de validation des outputs

C. Moteur d'Analyse par Modèle de Langage

1) *Architecture du Prompt Système*: Le système utilise un prompt d'ingénierie avancé définissant le rôle et les capacités de l'analyste IA :

[colback=blue!5!white, colframe=blue!75!black, title=Rôle de l'Analyste IA] **Senior Quantitative Financial Analyst** spécialisé dans :

- Trading d'actions et reconnaissance de patterns avancés
- Analyse technique (tendances, support/résistance, momentum)
- Évaluation de la confiance des modèles prédictifs
- Analyse de sentiment fondamental multi-sources
- Gestion des risques et dimensionnement des positions

2) *Framework d'Analyse Multi-Dimensionnel*: L'analyste IA évalue cinq dimensions principales :

a) 1. Analyse Technique:

- Identification de tendances (haussière, baissière, consolidation)
- Niveaux de support et résistance
- Évaluation du momentum et de la volatilité

b) 2. *Confiance dans les Modèles Prédicatifs:*

- Direction prévue par l'ensemble Prophet-XGBoost
- Magnitude et cohérence des prédictions
- Écart entre les modèles individuels

c) 3. *Analyse de Sentiment Fondamental:* Le système fournit à l'IA les métriques brutes pour découverte de patterns :

TABLE VI
MÉTRIQUES DE SENTIMENT ET SEUILS CRITIQUES

Métrique	Seuil Critique	Interprétation
sentiment_discord	> 0.4	Incertitude élevée, volatilité
fear_index	> 3	Panique, signal contrarien
product_hype_score	> 0	Catalyseurs fondamentaux
viral_sentiment	> 0.2 / < -0.2	Haussier / Baissier
attention_acceleration	> 1.0	Événement viral

d) 4. *Découverte de Patterns Cachés:* L'IA est explicitement programmée pour détecter :

- **Patterns séquentiels** : Pics de peur suivis de rebonds
 - **Divergences** : Sentiment haussier + prix baissiers = divergence haussière
 - **Anomalies de corrélation** : Relations inhabituelles entre métriques
 - **Changements de régime** : Shifts soudains dans plusieurs indicateurs
 - **Indicateurs avancés** : Métriques prédisant les mouvements de prix
 - **Corrélations décalées** : Sentiment d'aujourd'hui → prix de demain
- e) 5. *Évaluation des Risques:*
- Identification du régime de marché (tendance vs. mean-reversion)
 - Dimensionnement de position basé sur le niveau de confiance
 - Facteurs de risque spécifiques au contexte

3) *Format de Sortie Structuré:* Le système impose un format strict pour garantir l'actionnabilité :

```
RECOMMENDATION: [BUY/SELL/HOLD]
POSITION SIZE:
  [Conservative/Moderate/Aggressive] - [%
    of portfolio]
CONFIDENCE: [Low/Medium/High]

PRICE TARGETS:
- Entry Price: $XXX.XX
- Target Price (3-day): $XXX.XX
- Stop Loss: $XXX.XX

JUSTIFICATION:
[3-5 bullet points bases sur les donnees]

HIDDEN PATTERNS DISCOVERED:
[Patterns non-evidents dans les donnees
brutes]

RISK FACTORS:
[2-3 risques cles]
```

```
TIME HORIZON: [Short-term (1-3 days) /
Medium-term (1-2 weeks)]
```

Listing 2. Structure de Recommandation Obligatoire

D. *Préparation des Données pour l'IA*

1) *Formatage des Données Historiques:* Le système charge et formate les 14 derniers jours de données de prix :

```
def format_price_history(df):
    summary = "HISTORICAL PRICE DATA (Last 14
    Days):\n"

    for _, row in df.iterrows():
        date =
        row['timestamp'].strftime('%Y-%m-%d')
        summary += f"{date}:
        Close=${row['Close']:.2f}, "
        summary += f"High=${row['High']:.2f}, "
        summary += f"Low=${row['Low']:.2f}, "
        summary +=
        f"Volume={row['Volume']:,}.0f)\n"

    summary += f"\nPeriod Statistics:\n"
    summary += f"  - Average Close:
        ${df['Close'].mean():.2f}\n"
    summary += f"  - High:
        ${df['High'].max():.2f}\n"
    summary += f"  - Low:
        ${df['Low'].min():.2f}\n"
    price_change = ((df['Close'].iloc[-1] /
        df['Close'].iloc[0] - 1) *
        100)
    summary += f"  - Price Change:
        {price_change:.2f}%\n"
    summary += f"  - Volatility:
        ${df['Close'].std():.2f}\n"

    return summary
```

Listing 3. Extraction des Statistiques de Prix

2) *Présentation des Métriques Fondamentales:* Le formatage inclut trois couches d'information :

- 1) **Données Brutes** : Tableau complet des 14 derniers jours
 - Permet la découverte de patterns par l'IA
 - Format tabulaire pour faciliter l'analyse séquentielle
- 2) **Métriques Actuelles** : Valeurs du dernier jour avec interprétation

```
LATEST METRICS (2026-01-22):
- Sentiment Discord: 0.3247 (CONSENSUS)
- Fear Index: 2 (CALM)
- Product Hype Score: 5 (Strong
  Fundamental Focus)
- Viral Sentiment: 0.4521 (BULLISH)
- Post Volume: 142
- Attention Acceleration: 0.87 (Normal)
```

Listing 4. Exemple de Métriques avec Contexte

3) **Statistiques Agrégées** : Résumé sur 14 jours

- Min, max, moyenne pour chaque métrique

- Analyse de tendance (semaine récente vs. semaine précédente)
- Matrice de corrélations pour découverte de patterns

3) *Corrélations pour Pattern Discovery*: Le système calcule automatiquement les corrélations clés :

```
if len(df) >= 7:
    summary += f"\nCORRELATION PATTERNS:\n"
    corr_fear = df['fear_index'].corr(df['viral_sentiment'])
    summary += f" - Fear vs Sentiment: {corr_fear:.3f}\n"
    corr_discord = df['sentiment_discord'].corr(df['post_volume'])
    summary += f" - Discord vs Volume: {corr_discord:.3f}\n"
    corr_hype = df['product_hype_score'].corr(df['viral_sentiment'])
    summary += f" - Hype vs Sentiment: {corr_hype:.3f}\n"
    corr_accel = df['attention_acceleration'].corr(df['viral_sentiment'])
    summary += f" - Accel vs Sentiment: {corr_accel:.3f}\n"
```

Listing 5. Calcul des Corrélations Inter-Métriques

Ces corrélations permettent à l'IA d'identifier :

- Relations inversées (ex: Fear Index négatif → opportunité d'achat)
- Indicateurs confirmateurs (ex: Hype + Sentiment → mouvement fort)
- Signaux d'alerte précoce (ex: Discord élevé → volatilité imminente)

E. Invocation du Modèle de Langage

1) *Configuration Groq LLM*: Le système utilise LLaMA 3.3 70B via l'API Groq pour des raisons de performance :

```
from langchain_groq import ChatGroq

llm = ChatGroq(
    temperature=0.1, # Faible pour consistance
    model_name="llama-3.3-70b-versatile",
    groq_api_key=groq_api_key
)
```

Listing 6. Configuration du LLM

Justification du choix :

- **Température 0.1** : Maximise la cohérence et la reproductibilité
- **70B paramètres** : Capacité de raisonnement complexe multi-étapes
- **Groq** : Latence ultra-faible (1s), crucial pour trading temps réel

2) *Structure des Messages*:

```
messages = [
    SystemMessage(content=create_analyst_prompt()),
```

```
HumanMessage(content=f"""\Analyze NVIDIA
stock data:

{format_price_history(price_data)}
{format_predictions(predictions)}
{format_fundamental_analysis(fundamentals)}

ADDITIONAL CONTEXT:
- Current Date: {datetime.now().strftime('%Y-%m-%d')}
- Stock: NVIDIA Corporation (NVDA)
- Analysis Type: Quantitative Short-term Trading

Provide your complete analysis now.""")
]

response = llm.invoke(messages)
```

Listing 7. Construction du Contexte

F. Système de Distribution Automatisé

1) *Intégration Telegram*: Le système envoie deux types de notifications :

a) 1. *Notifications de Progression*:

```
start_msg = f"START NVDA Daily Pipeline
Started\n\n"
start_msg += f"DATE {datetime.now().strftime('%Y-%m-%d')}\n\n"
start_msg += f"Running:\n"
start_msg += f"1. Fundamental Analysis\n"
start_msg += f"2. Stock Forecasting\n"
start_msg += f"3. AI Recommendation"
send_telegram_message(start_msg,
    parse_mode='Markdown')
```

Listing 8. Notification de Démarrage

b) 2. *Rapport Final Complet*:

```
summary = f"CHECK NVDA Daily Analysis
Complete\n\n"
summary += f"CLOCK Duration: {pipeline_duration:.0f}s\n\n"
summary += f"CHART *Market Data:* \n"
summary += f"DOLLAR Price: ${current_price:.2f}\n"
summary += f"UP 14-Day: {price_change:+.2f}%\n"
summary += f"CRYSTAL 3-Day: ${predictions[-1]:.2f}\n\n"
summary += f"DOCUMENT Full report attached below"

send_telegram_message(summary,
    parse_mode='Markdown')

# Conversion en PDF
pdf_path = f"{os.path.splitext(filename)[0]}_report.pdf"
convert_md_to_pdf(filename,
    output_filename=pdf_path)
send_telegram_document(pdf_path, caption="AI
Report")
```

Listing 9. Génération du Résumé Final

G. Ordonnanceur Automatique

1) *Implémentation du Scheduler*: Le système utilise la bibliothèque `schedule` pour l'exécution quotidienne :

```
def start_scheduler(run_time="09:00"):
    # Verification de la configuration
    if not all([groq_api_key,
               telegram_bot_token,
               telegram_chat_id]):
        raise ConfigurationError("Missing API keys")

    # Verification scripts
    if not Path(FUNDAMENTAL_SCRIPT).exists():
        raise FileNotFoundError(
            f"{FUNDAMENTAL_SCRIPT} not found"
        )

    if not Path(FORECASTING_SCRIPT).exists():
        raise FileNotFoundError(
            f"{FORECASTING_SCRIPT} not found"
        )

    # Notification
    startup_msg = "START *NVDA AI Pipeline Bot*\n\n"
    startup_msg += f"CLOCK Daily at {run_time}\n"
    startup_msg += "CHART Full automation\n"
    startup_msg += "ROBOT Powered by Groq LLM"
    send_telegram_message(startup_msg,
                          parse_mode='Markdown')

    # Planification
    schedule.every().day.at(run_time).do(
        run_complete_pipeline
    )

    # Boucle
    while True:
        schedule.run_pending()
        time.sleep(60)
```

Listing 10. Configuration du Scheduler

2) *Gestion des Erreurs en Production*: Chaque phase du pipeline inclut une gestion d'erreur robuste :

```
def run_python_script(script_path,
                      script_name):
    try:
        result = subprocess.run(
            [sys.executable, script_path],
            capture_output=True,
            text=True,
            timeout=600 # 10 minutes
        )

        if result.returncode == 0:
            return True, result.stdout, ""
        else:
            return False, result.stdout,
            result.stderr

    except subprocess.TimeoutExpired:
        msg = f"{script_name} timed out"
        return False, "", msg
```

```
except Exception as e:
    msg = f"Error: {str(e)}"
    return False, "", msg
```

```
# Gestion des echecs
if not fund_success:
    error_msg = "CROSS *Pipeline Failed*\n\n"
    error_msg += "Fundamental Analysis failed\n\n"
    error_msg += f"Error: {fund_error[:500]}"
    send_telegram_message(error_msg,
                          parse_mode='Markdown')
    return False
```

Listing 11. Propagation des Erreurs

H. Exemple de Sortie Complète

1) *Structure du Rapport Généré*: [colback=green!5!white, colframe=green!75!black, title=Exemple de Recommandation IA] **RECOMMENDATION: BUY**

POSITION SIZE: Moderate - 15% of portfolio

CONFIDENCE: High

PRICE TARGETS:

- Entry Price: \$178.50
- Target Price (3-day): \$183.02
- Stop Loss: \$175.00

JUSTIFICATION:

- Ensemble model predicts +2.5% upside with high confidence (MAPE ;1%)
- Viral sentiment strongly bullish (0.45), product hype score elevated (5 mentions)
- Technical support at \$177 holding firm, bullish momentum confirmed
- Fear index low (2), indicating no panic selling pressure

HIDDEN PATTERNS DISCOVERED:

- Negative correlation between fear_index and next-day returns (-0.67)
- Product mentions precede price increases by 24-48h (lagged correlation)
- Attention acceleration spike detected 3 days ago, typical precursor to sustained rally

RISK FACTORS:

- Sentiment discord rising (0.32→0.41), potential volatility ahead
- 14-day price change +8.2%, possible short-term overbought
- Prediction variance between Prophet and XGBoost widening

TIME HORIZON: Short-term (1-3 days)

I. Métriques de Performance du Système

1) *Temps d'Exécution*:

2) *Fiabilité et Disponibilité*: Basé sur 30 jours de production :

- **Taux de succès global** : 96.7% (29/30 exécutions)
- **Échecs dus à timeout** : 1 cas (prévision XGBoost)
- **Échecs de notification** : 0 cas

TABLE VII
DURÉE MOYENNE DES PHASES DU PIPELINE

Phase	Durée Moyenne	Max Observé
Analyse Fondamentale	45 s	120 s
Prévision Stock	180 s	480 s
Analyse IA (LLM)	8 s	15 s
Génération PDF	3 s	5 s
Envoi Telegram	2 s	4 s
Total Pipeline	238 s	624 s

- **Temps d'indisponibilité** : 0 minutes (architecture stateless)

J. Avantages et Limitations

1) Avantages Clés:

- **Intégration Multi-Modale** : Combine prix, volume, sentiment, prédictions
- **Découverte Automatique de Patterns** : L'IA détecte des corrélations non évidentes
- **Recommandations Actionnables** : Format structuré avec prix d'entrée/sortie
- **Automatisation Complète** : Zero intervention humaine après configuration
- **Notification Temps Réel** : Alertes instantanées via Telegram
- **Traçabilité** : Chaque décision documentée avec justifications

2) Limitations Techniques:

- **Horizon Court Terme** : Prédictions limitées à 3 jours (contrainte des modèles)
- **Dépendance API** : Requiert connectivité Groq et Telegram
- **Absence de Backtesting** : Les recommandations ne sont pas validées historiquement
- **Biais de Récence** : Focus sur 14 derniers jours peut manquer tendances long terme
- **Coût des Tokens** : 15,000 tokens par analyse (Groq gratuit actuellement)

K. Perspectives d'Amélioration

1) Court Terme:

1) Système de Backtesting

- Simuler les recommandations sur données historiques
- Calculer Sharpe ratio, drawdown maximal, win rate
- Ajuster les seuils de confiance basés sur performance réelle

2) Gestion Multi-Actifs

- Étendre à un portefeuille de 5-10 actions tech
- Allocation dynamique basée sur corrélations
- Détection d'opportunités relatives (pairs trading)

3) Intégration News API

- Incorporer Reuters, Bloomberg en temps réel
- Détection d'événements (earnings, annonces produits)
- Ajustement des prédictions selon impact news

2) Long Terme:

1) Apprentissage par Renforcement

- Agent RL apprenant des résultats de recommandations
- Optimisation continue des poids ensemble
- Adaptation automatique aux changements de régime

2) Analyse de Sentiment Multilingue

- Intégrer forums chinois, japonais (Baidu, Weibo)
- Détection de FUD/FOMO cross-cultural
- Signaux précoces de marchés asiatiques

3) Interface Web Interactive

- Dashboard avec visualisations temps réel
- Simulation de stratégies (what-if analysis)
- Historique des recommandations et tracking de performance

XVII. VALIDATION ET ÉVALUATION

L'objectif de cette section est de documenter la validation de bout en bout, sans prétendre à des résultats de performance universels (les métriques dépendent des machines et des APIs).

A. Validation end-to-end

Nous validons le fonctionnement global en suivant le chemin critique :

- Les producers publient dans Kafka (vérification de l'existence des topics et consommation des derniers messages).
- Spark consomme les topics, parse le JSON et écrit dans MongoDB (vérification de l'insertion continue et présence du champ `timestamp`).
- Le dashboard lit les collections et affiche les KPI/charts (vérification des agrégations et de la cohérence temporelle).

B. Qualité des données

Les contrôles suivants sont utiles :

- **Schema validation** : absence de champs nuls inattendus et cohérence de type (ex. Volume en long).
- **Temporal consistency** : conversion correcte des dates en `timestamp` et tri temporel stable.
- **Duplicate handling** : en cas de redémarrage, inspection de doublons potentiels selon le sink et les offsets.

C. Observations de fonctionnement

En pratique, le découplage Kafka → Spark rend le système robuste aux variations de débit : les sources sociales peuvent produire par rafales, tandis que le stream processing de MongoDB sert de coussin analytique : le dashboard ne dépend pas directement de Kafka.

D. Limites de l'évaluation

Cette version ne comporte pas de benchmark complet (latency/throughput). Une amélioration consiste à instrumenter le pipeline (metrics Spark, consumer lag Kafka, temps d'écriture MongoDB) et à tracer ces métriques dans le dashboard ou un outil dédié.

XVIII. LIMITES ET PERSPECTIVES

Les limites actuelles incluent : contraintes d'API/taux (scraping), dérive potentielle des schemas de données, sentiment rudimentaire. Les pistes d'amélioration : classification du sentiment avec modèles pré-entraînés, enrichissements de features (événements macro, résultats), alertes en temps réel, tests d'intégration et scalability (Kafka partitioning, Spark tuning).

XIX. CONCLUSION

Nous avons présenté un pipeline de données temps réel pour Nvidia, depuis l'ingestion multi-sources avec Kafka jusqu'à la visualisation avec Streamlit, en passant par le traitement en flux avec Spark et le stockage dans MongoDB. La solution facilite l'analyse technique et contextuelle, et constitue une base solide pour des extensions orientées sentiment, alertes et scalabilité.

REMERCIEMENTS

Nous remercions notre encadrant **Dr Yasser Etmadani Elalami** pour son accompagnement et ses conseils tout au long du projet, ainsi que l'équipe pédagogique. Nous remercions également les communautés open-source (Kafka, Spark, MongoDB, Streamlit) dont les outils ont rendu ce travail possible.

REFERENCES

- [1] Apache Kafka Documentation. <https://kafka.apache.org/documentation/>
- [2] Apache Spark Structured Streaming Programming Guide. <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>
- [3] MongoDB Spark Connector Documentation. <https://www.mongodb.com/docs/spark-connector/current/>
- [4] MongoDB Atlas Documentation. <https://www.mongodb.com/docs/atlas/>
- [5] Streamlit Documentation. <https://docs.streamlit.io/>
- [6] yfinance Documentation (Yahoo Finance data). <https://pypi.org/project/yfinance/>
- [7] Docker Compose Overview. <https://docs.docker.com/compose/>
- [8] Confluent Platform Images (cp-zookeeper, cp-kafka). <https://hub.docker.com/r/confluentinc/cp-kafka>
- [9] Reddit API Documentation. <https://www.reddit.com/dev/api/>
- [10] StockTwits API Documentation. <https://api.stocktwits.com/developers/docs>